UNIT 5

Q1) Post Correspondence Problem (PCP)

The **Post Correspondence Problem (PCP)** is an important decision problem in the theory of computation that deals with matching sequences of strings. It is defined as follows:

Problem Statement:

Given two lists of strings over some alphabet, $A = \{a_1, a_2, \ldots, a_n\}$ and $B = \{b_1, b_2, \ldots, b_n\}$, find a sequence of indices $i_1, i_2, \ldots, i_k$ (where $i_j \in \{1, 2, \ldots, n\}$) such that: $a_{i_1} a_{i_2} \ldots a_{i_k} = b_{i_1} b_{i_2} \ldots b_{i_k}$ If such a sequence exists, the PCP has a solution; otherwise, it does not.

Key Features:

- PCP is **undecidable**, meaning there is no general algorithm that can determine whether a solution exists for every instance of the problem.
- It is used to prove the undecidability of other problems in computation theory.

Example:

Let $A = \{a, ab, b\}$ and $B = \{aa, b, ba\}$.
The solution is the sequence of indices $(1, 2, 3)$, because: $a \cdot ab \cdot b = aa \cdot b \cdot ba$

Q2) Differences between Pushdown Automata (PDA) and Turing Machine (TM)

Aspect	Pushdown Automata (PDA)	Turing Machine (TM)
Definition	PDA is an automaton with a stack for memory.	TM is a computational model with an infinite tape as memory.
Power	Recognizes **context-free languages (CFLs)**.	Recognizes **recursively enumerable languages (RELs)**.
Memory	Utilizes a single stack for storage, allowing Last-In-First-Out (LIFO) access.	Uses an infinite tape with unrestricted read/write access.
Transitions	Based on input symbol, current state, and top of the stack.	Based on input symbol, current state, and tape contents.
Acceptance	By final state or empty stack.	By halting in an accepting state.
Direction of Movement	Reads input left-to-right.	Can move left or right on the tape, and stay on the same cell.
Language Recognition	Limited to simpler languages (CFLs like balanced parentheses).	Can simulate any algorithm, capable of solving complex problems.
Decidability	Decidable problems only.	Can handle undecidable problems but may not always halt.

Q3) Variants of Turing Machine

5.7 Variants of Turing Machine

SPPU : May-11,16,18, Dec.-10,13,15,16,17,18, Mark

5.7.1 Multitape Turing Machine

The multitape turing machine is a type of turing machine in which there are more than one input tapes. Each tape is divided into cells and each cell can hold any symbol of finite tape alphabet. The multitape turing machine is as shown in the Fig. 5.7.1.

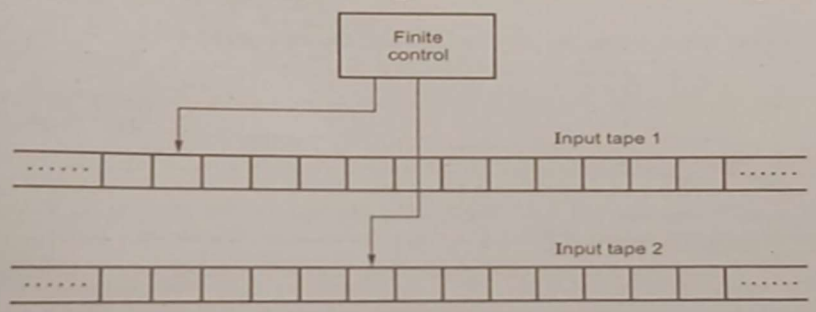


Fig. 5.7.1 A multitape turing machine

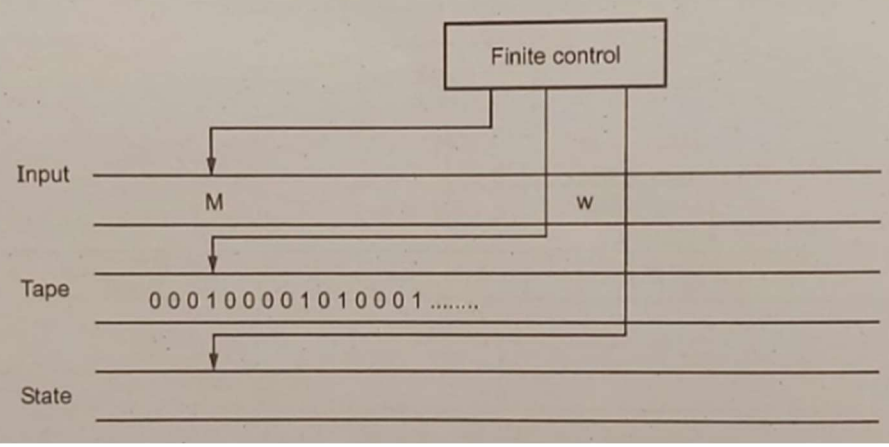
This TM is more powerful than the basic turing machine. Because finite control reads more than one input tape and more symbols can be scanned at a time.

5.7.2 Universal Turing Machine

SPPU : Dec.-10, May-11, Marks 8

- The universal language L_u is a set of binary strings which can be modeled by a turing machine.
- The universal language can be represented by a **pair (M, w)** where M is a TM that accepts this languages and w is a binary string in$(0 + 1)^*$ such that w belongs to L(M). Thus we can say that any binary string belongs to a universal language.

- The **universal language** can be represented by $L_u = L(U)$ where U is a universal Turing Machine.
- In fact U is a binary string. This binary string represents various codes of many Turing machines.
- Thus the universal turing machine is a Turing machine which accepts many Turing machines.
- The TM U is a multitape Turing machine which can be as shown below.
- The universal turing machine U accepts the TM M.
- The transitions of M are stored initially on first tape along with the string w.
- On the second tape the simulated tape of M is placed. Here the tape symbol X_i of M will be represented by 0^i and tape symbols are separated by single 1's.
- On the third tape various states of machine M are stored. The state q_i is represented by i 0's.



Q4) **Church's Turing thesis**

Turing machine is defined as an abstract representation of a computing device such as hardware in computers. Alan Turing proposed Logical Computing Machines (LCMs), i.e. Turing's expressions for Turing Machines. This was done to define algorithms properly. So, Church made a mechanical method named as 'M' for manipulation of strings by using logic and mathematics. This method M must pass the following statements:

- Number of instructions in M must be finite.
- Output should be produced after performing finite number of steps.
- It should not be imaginary, i.e. can be made in real life.
- It should not require any complex understanding.

Using these statements Church proposed a hypothesis called
that can be stated as: "The assumption that the intuitive notion of computable functions can be identified with partial recursive functions."

Or in simple words we can say that "Every computation that can be carried out in the real world can be effectively performed by a Turing Machine."

In 1930, this statement was first formulated by Alonzo Church and is usually referred to as Church's thesis, or the Church-Turing thesis. However, this hypothesis cannot be proved. The recursive functions can be computable after taking following assumptions:

1. Each and every function must be computable.
2. Let 'F' be the computable function and after performing some elementary operations to 'F', it will transform a new function 'G' then this function 'G' automatically becomes the computable function.
3. If any functions that follow above two assumptions must be states as computable function.

Q5)Write short notes on i) Unsolvable problems ii) Applications of Turing Machine

i) Unsolvable Problems

Unsolvable problems are problems for which no algorithm or computational procedure can provide a solution for all possible inputs. These problems are fundamental to understanding the limits of computation.

Key Features:

1. **Definition**: An unsolvable problem cannot be solved by any Turing Machine, meaning it is undecidable.
2. **Halting Problem**: The most famous example, introduced by Alan Turing, asks whether a Turing Machine will halt on a given input. Turing proved that no algorithm can solve this problem for all inputs.
3. **Rice's Theorem**: States that all non-trivial properties of the language recognized by a Turing Machine are undecidable.
4. **Examples**:
    - Determining whether a Diophantine equation has a solution (Hilbert's 10th Problem).
    - Checking equivalence of two arbitrary programs.

Importance:

Unsolvable problems demonstrate the inherent limitations of computation and emphasize the need for approximations or heuristics in certain areas.

ii) Applications of Turing Machine

A **Turing Machine (TM)** is a theoretical computational model that plays a foundational role in computer science. Despite being abstract, it has significant applications:

Applications:

1. **Foundation of Computation Theory**:
   - TM formalizes the concept of algorithms and serves as a benchmark for defining computable functions.
   - Helps distinguish between decidable and undecidable problems.
2. **Programming Language Design**:
   - TMs influence the design of programming languages by modeling language constructs and control flows.
   - Serve as a theoretical framework for understanding compiler design.
3. **Artificial Intelligence and Machine Learning**:
   - TM concepts guide the development of algorithms, especially those mimicking step-by-step human reasoning.
4. **Automata Theory and Formal Languages**:
   - TM is used to define recursively enumerable languages.
   - Forms the basis for studying regular, context-free, and context-sensitive grammars.
5. **Cryptography**:
   - Provides insights into problems like one-way functions and their relation to computational complexity.
6. **Complexity Theory**:
   - TM is used to classify problems based on computational complexity classes (e.g., P, NP, NP-complete).

Example: A Universal Turing Machine simulates other Turing Machines, demonstrating the capability of general-purpose computation, analogous to modern programmable computers.

---

q6) Recursive and Recursively Enumerable Languages

1. Recursive Languages (RLs):

Definition: A language LLL is **recursive** if there exists a Turing Machine (TM) that:

1. **Halts** on every input, regardless of whether the input is in the language.
2. **Accepts** the input if it belongs to the language LLL.
3. **Rejects** the input if it does not belong to LLL.

Key Characteristics:

The TM always produces a definite answer (accept or reject).

Recursive languages are also called **decidable languages** because a decision procedure exists for them.

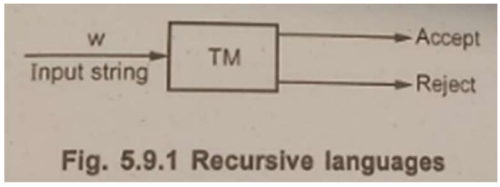


Fig. 5.9.1 Recursive languages

2. Recursively Enumerable Languages (RELs):

Definition: A language LLL is **recursively enumerable** if there exists a Turing Machine (TM) that:

1. Accepts the input if it belongs to the language LLL.

2. May run indefinitely if the input does not belong to LLL.

Key Characteristics:

The TM is not required to halt for inputs that are not in the language.

All recursive languages are also recursively enumerable, but the converse is not true.

RELs are also called **semi-decidable languages** because the TM provides an answer only for inputs in LLL.

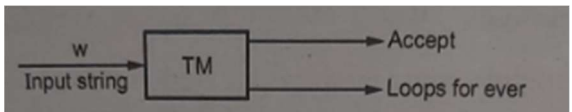


Fig. 5.9.2 Recursively enumerable languages

---

q7)Halting Problem

5.11 Halting Problem of TM

SPPU : Dec-16,17, May-19, Marks 8

To state halting problem we will consider the given configuration of a turing machine. The output of TM can be

i) **Halt :** The machine starting at this configuration will halt after a finite number of states.

ii) **No Halt :** The machine starting at this configuration never reaches a halt state, no matter how long it runs.

- Now the question arises based on these two observation : Given any functional matrix, input data tape and initial configuration, then is it possible to determine whether the process will ever halt? This is called **halting problem**.
- That means we are asking for a procedure which enable us to solve the halting problem for every pair (machine, tape). The answer is "no".
- That is the halting problem is unsolvable.
- Now we will prove why it is unsolvable.
- Let, there exists a TM M_1 which decides whether or not any computation by a TM T will ever halt when a description d_T of T and tape t of T is given [That means the input to machine M_1 will be (machine, tape) pair].
- Then for every input (t, d_T) to M_1 if T halt for input t, M_1 also halts which is called **accept halt.**
- Similarly if T does not halt for input t then the M_1 will halt which is called **reject halt**. This is shown in Fig. 5.11.1.
- Now we will consider another Turing Machine M_2 which takes an input d_T.
- It first copies d_T and duplicates d_T on its tape and then this duplicated tape information is given as input to machine M_1. But machine M_1 is a modified machine with the modification that whenever M_1 is supposed to reach an accept halt, M_2 loops forever.

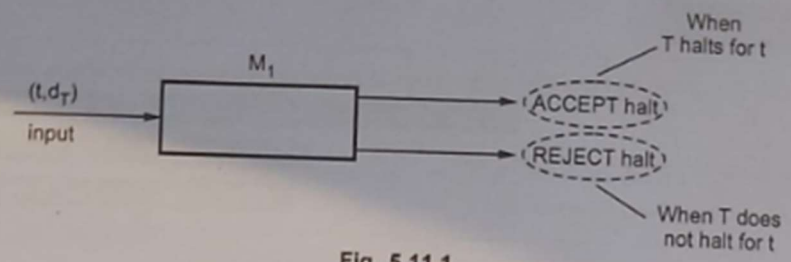


Fig. 5.11.1

- Hence behavior of M_2 is as given. It loops if T halts for input $t = d_T$ and halts if T does not halt for $T = d_T$.
- The T is any orbitary Turing machine.

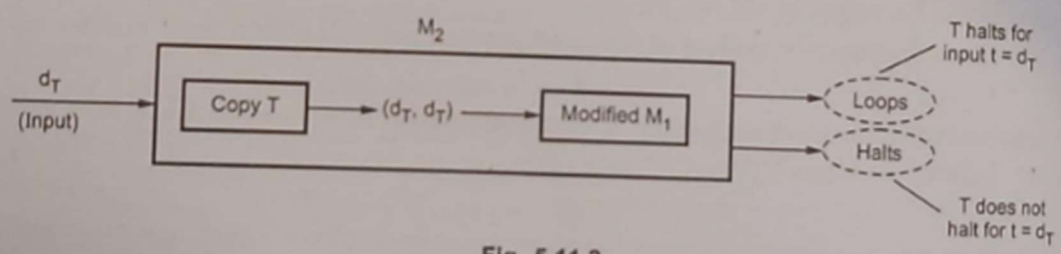


Fig. 5.11.2

As M_2 itself is one turing machine we will take M_2 = T. That means we will replace T by M_2 from above given machine.

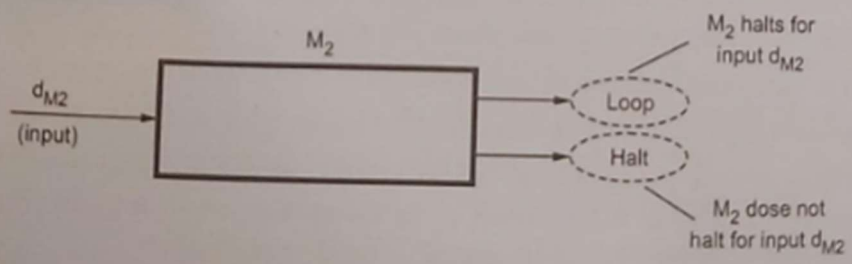


Fig. 5.11.3

Thus machine M_2 halts for input d_{M2} if M_2 does not halt for input d_{M2}. This is a contradiction. That means a machine M_1 which can tell whether any other TM will halt on particular input does not exist. Hence halting problem is unsolvable.